

AggreKV: Boosting Hash-based KV-SSD Performance via Locality-Centric Optimizations

Abstract—Emerging Key-Value Solid-State Drives (KV-SSDs) improve the performance and scalability of modern Key-Value (KV) stores by eliminating host-side software overheads. However, KV-SSDs suffer from severe mapping-induced performance overheads, as the efficiency of hash mapping is constrained by limited on-device DRAM. Specifically, mapping inflation and address randomization degrade the mapping cache hit ratio; partial-key mapping necessitates costly flash accesses for key verification; and logically scattered mapping entries trigger massive mapping page updates during garbage collection (GC). To overcome these multi-layer bottlenecks, we propose AggreKV, a novel locality-centric Key-to-Physical (K2P) mapping approach. AggreKV systematically reconstructs locality along three dimensions: first, **proactive** dual-granular mapping cache captures temporal locality and extracts hot entries to improve the space efficiency while preserving low metadata overhead; second, selective full-key indexing leverages key access locality to build a memory-efficient full-key cache that bypasses expensive flash-based verification; finally, locality-aware data placement enforces spatial locality by clustering data from adjacent mapping regions within the same **Flash Block Group (FBG)**, mitigating the overhead of cache misses during GC. Compared with state-of-the-art designs, AggreKV delivers average read and update throughput enhancement of **30.47%** and **23.95%**, while reducing tail latencies by up to **45%** and **27.34%**, respectively.

Index Terms—Flash memories, key-value solid-state drives, caching algorithms, storage management.

I. INTRODUCTION

KEY-VALUE stores (e.g., RocksDB [1]) offer high performance, massive scalability, and data flexibility, thus becoming important foundations for today's data-intensive applications. However, conventional key-value stores are built on top of the file system and block layer. The hierarchical storage software stack becomes heavy and causes performance and scalability bottlenecks, as the underlying NVMe SSDs offer high bandwidth and low latency. By offloading key-value management into the device, Key-Value SSDs (KV-SSDs) provide direct key-value access to the applications. Hence, the software overheads and thus the bottlenecks can be eliminated [2].

Constrained by limited internal hardware resources, KV-SSDs necessitate efficient mechanisms to manage Key-to-Physical (K2P) mappings for balancing resource consumption and device performance. Existing mapping architectures are primarily categorized into two approaches: LSM-trees [3], [4] and hash tables [5], [6], [7], [8]. Compared to LSM-tree approaches, hash-based KV-SSDs deliver better read performance while incurring lower computational overhead, which enables seamless integration into existing hardware platforms. Consequently, this paper focuses primarily on hash-based KV-SSDs.

However, as SSD capacity and the volume of KV pairs scale, K2P mapping management becomes a critical performance bottleneck for hash-based KV-SSDs. Due to severe DRAM memory shortage, the K2P mapping introduces substantial performance overheads on the I/O path. As analyzed in detail in Section III, we identify three overhead sources: address translation before data access, key verification during KV updates, and mapping maintenance during write-induced GC.

First, fine-grained and randomized K2P hash mapping causes high address translation overhead. To reduce on-device DRAM overhead while minimizing extra flash reads for mapping pages on the critical I/O path, conventional **Block-SSDs** cache recently accessed mapping pages, rather than mapping entries [9], [10]. As each mapping page contains contiguous mapping entries, page-based caching can exploit spatial locality (via prefetching) to improve cache hit ratio and simplify cache management by avoiding managing a large number of fine-grained entries. However, fine-grained KV pairs result in a significantly inflated hash mapping table (e.g., an SSD filled with 1KB KV pairs requires $4\times$ as many mapping entries as one utilizing 4KB pages). Furthermore, hash indexing inherently randomizes the logical distribution of mapping entries. This randomization substantially disrupts the spatial locality of data access, transforming it into random reads and updates of mapping entries, thereby weakening the efficiency of page-based caching. Consequently, **our experiment shows** the cache hit ratio of KV-SSD mapping table is as low as 36.34%, leading to a 49.33% performance degradation.

Second, key verification imposes significant read-before-write overhead on update operations. Due to constrained DRAM capacity, the hash mapping table keeps partial fields of keys (i.e., fingerprint), instead of complete keys. To process a KV read/update request, the mapping table is looked up to obtain the storage location of target KV pair. However, a false key hit may occur, since two different keys may share the same fingerprint in the mapping table. Hence, key verification is required, where the KV-SSD compares the target key with the complete key on the flash memory before each data access. The performance overhead of verification is tolerable for reads, since the complete key is retrieved together with data during a single flash read, and hash collisions rarely happen. But for updates, regardless of hash collisions, it incurs an expensive read-before-write into the critical I/O path. This not only occupies the bandwidth of the flash memory but also prolongs the update process severely. Our experiment shows that the key verification causes 25.74% update throughput loss, and increases the average latency by 21.29%.

Third, GC introduces periodical data migrations, leading to high mapping update overhead. To accommodate

the out-of-place update nature of flash memory, SSD performs Garbage Collection (GC) to reclaim space occupied by invalid data periodically. This process operates at the granularity of Flash Block Groups (FBGs), where all valid data within a victim FBG must be migrated to a reserved area prior to block erasure. However, the inherent randomness of hash indexing in KV-SSDs scatters the mapping entries of a single FBG across the entire mapping table. Relocating vast number of valid KV pairs accommodated in an FBG triggers extensive and highly dispersed mapping updates, drastically reducing the mapping cache hit ratio. Consequently, the efficiency of GC in KV-SSDs is severely compromised. As demonstrated in our evaluations, this inefficiency yields up to $7\times$ more mapping misses and a 159.4% degradation in tail latency compared to conventional Block-SSDs.

Based on our study, we propose AggreKV, a novel hash-based K2P mapping scheme that employs a locality-centric design principle to improve overall performance of KV-SSD. AggreKV relies on three key techniques:

First, to address the address translation overhead caused by fine-grained and randomized hash mapping, we propose **Proactive Dual-Granular Mapping Cache**. It precisely captures temporal locality by dynamically extracting only active mapping entries at a fine granularity, maximizing mapping hit ratio while preserving as low metadata maintenance overhead as page-based caching.

Second, to alleviate key verification overhead on updates, we design **Selective Full-Key Indexing**. It leverages the key length characteristics of real-world workloads, rather than imposing a loose limit on the key length that prevents maintaining complete keys within constrained memory. Specifically, it selectively preserves short full keys with high update locality and prefetches keys upon misses, thereby successfully bypassing the expensive flash key verification during data updates.

Third, to mitigate GC mapping update overhead due to massive randomized mapping entries related to a single FBG, we develop **Locality-Aware Data Placement**. It utilizes hardware support of multiple active FBGs, reconstructs spatial locality of mapping updates by physically aggregating KV pairs corresponding to adjacent mapping entries into the same FBG. Therefore, GC migrations produce more clustered mapping updates, thereby improving cache efficiency and minimizing the overhead of mapping maintenance.

To evaluate the capability of AggreKV, a series of experiments were conducted. The results show that the proposed design significantly enhanced KV-SSD performance and reduced access latency with negligible hardware resource consumption. Specifically, it achieves a 20.01% improvement of the mapping-table hit ratio, boosting the read and update throughput by 30.47% and 23.95% while effectively reducing the tail latency by up to 45% and 27.34% respectively, compared to state-of-the-art KV-SSD mapping designs.

II. BACKGROUND

A. NAND Flash-based KV-SSD

NAND flash memory serves as the base storage medium for modern SSDs. Characterized by its non-volatile nature and

unique architecture, data must be erased in large blocks before being programmed in smaller pages. Because NAND flash lacks the ability to overwrite data in-place and possesses a finite endurance limited by program-erase cycles, it requires a sophisticated internal controller to manage operations via a Flash Translation Layer (FTL). The conventional FTL in a Block-SSD is responsible for Logical Page Number (LPN) to Physical Page Number (PPN) mapping, wear leveling, and garbage collection, presenting the flash medium to the host as a contiguous array of fixed-size logical blocks. The core data structure for the FTL is typically a mapping table that uses the LPN as an index to point to the corresponding PPN.

Emerging KV-SSDs fundamentally eliminate the heavy abstraction overhead inherent in conventional storage stacks [11]. In a conventional I/O stack, variable-length KV pairs must undergo multiple redundant layers of mapping, from KV pairs to files, then to fixed-size logical blocks, and finally to physical flash pages, which results in severe CPU and memory contention. KV-SSD addresses these inefficiencies by simplifying the storage architecture, offloading host-side operations such as indexing, data packing, and iterator management directly to the device's FTL. However, the research on the efficiency of internal KV mapping still remains insufficient, hindering the widespread adoption of KV-SSDs. Therefore, a systematic characterization of the internal mechanisms within the KV-SSD is essential to identify required optimizations and fully unleash the performance potential.

B. Design of KV-SSD index

Conventional Block-SSDs employ a hierarchical memory architecture to manage the mapping table. Typically, they group and persist the table entries as mapping pages on flash, while allocating a DRAM copy to support better access performance. Given that each mapping entry requires only a 4B PPN, a DRAM ratio equivalent to 0.1% of the device logical capacity is sufficient to host the entire table. To further reduce DRAM costs, certain SSD designs implement an on-demand caching mechanism [9], [10], as illustrated in Figure 1(a). The locations of mapping pages are tracked by a DRAM-resident Global Translation Directory (GTD). Upon receiving an LPN request, the SSD first queries GTD to find the corresponding mapping page, then it retrieves the target PPN via an intra-page offset. To optimize performance, a subset of mapping pages is dynamically loaded into the DRAM, called Cached Mapping Table (CMT). Consequently, address translation requests that hits CMT safely bypass costly flash retrieval operations.

The architectural transition from conventional block-based storage to KV-SSD fundamentally relies on indexing of variable-length KV pairs. The design space is primarily defined by two architectures: the LSM-tree approach, typical of host-side software like RocksDB [1], and the hash approach, mainly utilized by the Samsung PM983 KV-SSD [6].

LSM-tree-based KV-SSDs manage mapping information using an LSM-tree, as illustrated in Figure 1(b). For PUT() operations, it first buffers incoming data into a DRAM-based Skiplist (L_0) and then flush it as immutable, sorted data segments to the flash. This fundamentally transforms random write patterns into append-only accesses that align with the

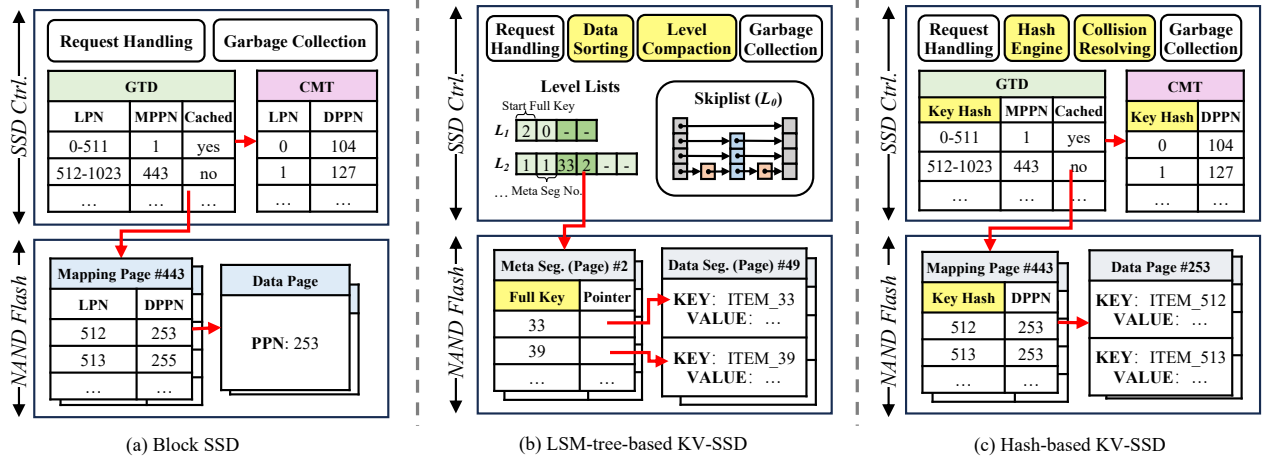


Fig. 1. Comparison among Different SSD Implementations

physical properties of NAND flash, leading to a better write performance. For `GET()` operations, the KV-SSD first checks the Skiplist. Upon a miss, it scans the Level Lists (L_n) from top to bottom to obtain the target physical data address. Limited by DRAM resources, the level lists are maintained in two parts: flash-based Meta Segments that contain full K2P mappings, and simplified DRAM-resident entries that store only a start key and page number of each segment.

Despite high write performance delivered by the append-only mechanism, LSM-tree-based KV-SSDs incur significant read latency and background maintenance overhead. First, data retrieval requires a layer-by-layer scan of level lists, causing up to n (i.e., the number of levels) additional flash reads in the worst case. Second, a periodic inter-level compaction is required to reclaim space and maintain efficiency. The compaction involves merging and sorting data, which incurs a severe on-device computational bottleneck and degrades long-term performance [4]. Furthermore, the compaction amplifies flash writes, thereby reducing the overall lifespan of the SSD.

Hash-based KV-SSDs adopt a hash index structure that closely resembles conventional **Block-SSDs**, as shown in Figure 1(c). For `GET()` operations, the hash engine first converts the key into a fixed-length hash value [12], [13]. Then, the prefix of this hash value serves as an offset within the GTD to locate the corresponding mapping page. Finally, the physical data address is retrieved from this mapping page using the remaining part of hash value. It also undergoes the same address translation process for `PUT()` operations. Unlike **Block-SSDs**, the hash mapping introduces the possibility of collisions (i.e., different keys share the same hash value). Consequently, the SSD must verify the original key before returning the data or updating the mapping entry. A mismatch triggers a collision resolution process (e.g., Linear Probing [14]) to continue the search until the target entry is identified.

Compared to LSM-tree approach, the hash-based KV-SSD features a more straightforward mapping logic. Under most circumstances, the read process requires only zero or one mapping page load, exhibiting high read performance. Leveraging existing hardware acceleration of hash algorithms in modern SSDs, the hash approach avoids additional compute-intensive operations, enabling seamless integration into current

hardware platforms. These advantages position hash-based KV-SSDs as an ideal development trajectory for read-intensive scenarios.

III. CHALLENGE AND MOTIVATION

However, the hash-based KV-SSD still faces significant performance challenges. First, storing a massive volume of fine-grained KV pairs yields a vast number of mapping entries. For example, a KV-SSD populated with 1KB KV pairs must manage $4\times$ number of mapping entries compared to one populated with 4KB pages. Second, to accelerate the filtering of hash collisions, each mapping entry in the KV-SSD introduces a fingerprint field [4], [8], which further increases memory overhead by up to $2\times$. The hash mapping table thus expands considerably and becomes too large to fit into the onboard DRAM, causing severe mapping-access inefficiency. This mapping-induced challenge manifests across three key stages: address translation, data updates, and garbage collection.

RHIK [8], the state-of-the-art hash-based KV-SSD, dynamically scales its mapping table based on the number of KV pairs to improve cache efficiency. However, it still suffers from severe table inflation and performance degradation when managing massive fine-grained KV mappings. In the subsequent experiments, we use RHIK to demonstrate the mapping-induced challenge across the three aforementioned stages.

A. Mapping Cache Inefficiency in Address Translation

To investigate the differences in mapping access efficiency between hash-based KV-SSDs and conventional **Block-SSDs**, we evaluated the mapping cache hit ratios and the distribution of mapping entries for both SSD types under random and sequential read workloads using an SSD simulator specified in §V-A. The KV-SSD is implemented based on RHIK [8].

In the experiments, each SSD is configured with 256GB logical storage capacity and 256MB on-board DRAM cache. **The mapping entry for the Block-SSD is 4B in size.** For the KV-SSD, each mapping entry is 8B, comprising a 4B physical address and a 4B fingerprint. Detailed hardware configurations are provided in §V-A. **To further evaluate the Block-SSD under the same number of cached mapping entries as the KV-SSD,** we introduce a configuration with a smaller DRAM cache,

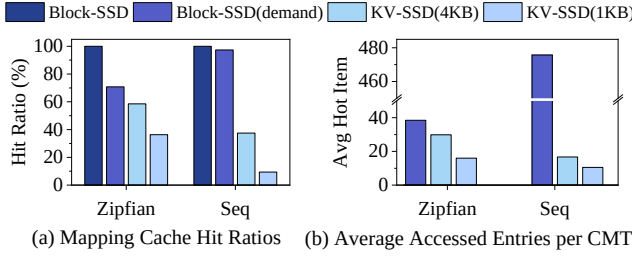


Fig. 2. Mapping Cache Hit Ratios and Average Number of Accessed Entries in Top-10% CMT for Block- and KV-SSD under Read Workloads.

denoted as “**Block-SSD (demand)**”. For the **Block-SSD**, the granularity of data access is 4KB. For the KV-SSD, two distinct granularity configurations are utilized: 4KB and 1KB KV pairs. The total size of the dataset is 256GB. The random pattern of data access follows a Zipfian distribution ($\alpha=1.0$).

As shown in Figure 2(a), the mapping hit ratio of KV-SSD is lower than that of **Block-SSD** across workloads. Specifically, **Block-SSD** exhibits a maximum hit ratio since its mapping table size is equal to the cache size. Under Zipfian workload, **Block-SSD (demand)** achieves hit ratio of 70.76%, whereas KV-SSD drops to 58.55%. The gap widens as KV-SSD with 1KB granularity achieves only a 36.34% hit ratio, which causes a 49.33% performance degradation compared to the full-cache **Block-SSD**. Under sequential workload, **Block-SSD** (demand) shows a significantly higher hit ratio of 97.32%, since most requests hit in the recent-loaded mapping page. However, KV-SSD performs worse than under Zipfian workload, yielding hit ratios of 37.49% and 9.39% for 4KB and 1KB requests, respectively. These behaviors are mainly caused by two factors: (1) hash mapping inherently randomizes the logical address distribution of mapping entries, which breaks spatial locality and makes page-level cache less effective; and (2) fine-grained KV mapping significantly inflates the number of mapping entries, which enlarges the mapping table footprint and further reduces hit ratio of the mapping cache.

Figure 2(b) further reveals the effect of the address randomization through intra-page cache utilization. Specifically, under Zipfian workload, the average number of accessed mapping entries in Top-10% CMT is 38.45 for **Block-SSD** (demand), 29.83 for KV-SSD with 4KB accesses, and 15.99 for KV-SSD with 1KB accesses. Under sequential workloads, the number of accessed mapping entries in **Block-SSD** (demand) substantially increases to 475.68, while the KV-SSD shows minimal variation (16.71 and 10.51 for 4KB and 1KB requests). These results demonstrate that the randomness of hash indexing disrupts the spatial locality of mapping entries. This disruption significantly reduces the number of valid entries within a single cached mapping page and severely degrades the efficiency of the page-based cache. Coupled with large key cardinality, the mapping cache pressure of KV-SSDs is substantially higher than that of conventional **Block-SSDs**.

Challenge #1: Mapping cache efficiency of KV-SSD is significantly reduced, because fine-grained mapping inflates the mapping table and the random nature of hash indexing breaks spatial locality of mapping entries.

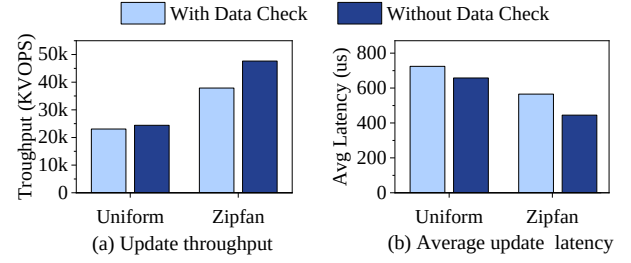


Fig. 3. Update Performance of KV-SSD with/without Key Verification.

Notably, although overall cache efficiency declines in the KV-SSD, the behavior of mapping cache still correlates with access skewness. Compared to sequential workload, which produces the same effect as uniform random workloads for hash-based KV-SSDs, the Zipfian workload witnesses higher mapping cache hit ratio by 21.06% and 26.95%. This phenomenon occurs because temporal locality remains largely intact, allowing the cache to capture frequently accessed entries and partially recover access efficiency. However, this improvement is constrained by the inefficiency of page-based caching, as frequently accessed hot entries are tightly coupled with cold entries within the same mapping page.

Prior **Block-SSD** mapping cache designs [15], [16] leverage spatial locality to prefetch mapping entries in page granularity and exploits temporal locality using simple LRU or similar policies to cache fine-grained mapping entries. However, they are insufficient for KV-SSDs. First, mapping accesses demonstrate no spatial locality in KV-SSDs due to hash indexing, rendering the sequential prefetching ineffective. Second, KV-SSDs maintain an inflated, massive number of mapping entries, requiring careful cache admission and memory-efficient fine-grained cache management to improve cache hit ratio.

Observation #1: Despite hash-induced spatial randomness, mapping accesses retain temporal locality; precisely isolating and caching hot entries from coarse-grained pages while maintaining a minimal metadata footprint is crucial for higher mapping cache efficiency.

B. Update Overhead from Partial-Key Mapping Verification

To limit DRAM overhead, each mapping entry of KV-SSD typically stores only a key fingerprint, rather than the complete key. Consequently, once a candidate entry is located, the SSD must verify the target key against the complete key in the flash to prevent false matches caused by hash collisions. This verification is particularly expensive for updates because it introduces an additional read-before-write operation, which consumes flash bandwidth and prolongs the write path.

To further determine the impact of key verification during updates, we compared the update performance on the same KV-SSD with/without key verification. The dataset size is 256GB, and the workload follows uniform and Zipfian distribution ($\alpha=1.0$) respectively. The results are illustrated in Figure 3.

Enabling key verification significantly degrades the update performance of the KV-SSD. Under uniform workload, the

throughput of the KV-SSD decreases by 5.72% and the average latency increases by 9.23%, because mapping cache misses dominate the overall overhead. As access skew increases, the mapping cache hit ratio improves, widening the performance gap caused by key verification: throughput drops by 25.74% and average latency rises by 21.29%. However, hash collisions rarely occur in our evaluation, exhibiting a probability of only 0.74%.

Challenge #2: Although hash collisions rarely happen, flash-based partial-key mapping verification still enforces expensive read-before-write operations on updates, significantly degrading throughput and latency.

To alleviate the I/O overhead of key verification, existing host-side KV stores [17] typically cache recently used keys, which relies on larger host memory. Conversely, the on-board DRAM within an SSD is highly restricted, posing a significant challenge for efficient key caching. Specifically, in commercial KV-SSDs, the maximum supported key length is typically 255B to ensure broad application compatibility [18]. This loose upper bound makes it impractical to retain complete keys for all entries in DRAM.

Prior measurements on real-world workloads show substantial variation in key and value sizes [19]. The length of data keys exhibits highly distinct characteristics across various datasets. Workloads such as dedup engines and social-graph services tend to use much shorter keys ($\sim 20B$), whereas online-gaming and blockchain-related workloads typically use longer keys (37–94B), yet none exceeds 100B. Since verification cost is roughly similar regardless of key length (i.e., one additional flash read), while shorter keys consume less memory, exploiting actual key lengths makes it feasible to accommodate full keys in DRAM. Consequently, this enables the SSD to capture the underlying key access locality, providing a practical path to bypass costly flash-based key verifications.

Observation #2: Real-world key-length diversity makes full-key caching practical within limited DRAM, enabling the KV-SSD to capture key access locality and mitigate flash-based key verification overhead.

C. GC Degradation from Inefficient Mapping Updates

The GC efficiency depends heavily on the mapping cache hit ratio, as data migration necessitates a substantial number of mapping updates. Specifically, the SSD retrieves mapping entries to identify obsolete data and updates the corresponding entries after relocating valid data. However, the randomness of hash indexing in KV-SSDs inherently scatters the mapping entries of written KV pairs across the entire mapping table. Since the volume of mapping updates depends on the massive number of valid fine-grained KV pairs migrated from the victim FBG, relocating these valid pairs triggers extensive and dispersed updates across a broad range of mapping pages. Consequently, cache misses occur frequently during GC in KV-SSDs, thereby severely degrading tail latency.

To analyze GC efficiency of KV-SSD, we compare long-term write performance of the Block-SSD and KV-SSD. The size of FBG is configured to 256MB, and the over-provisioning

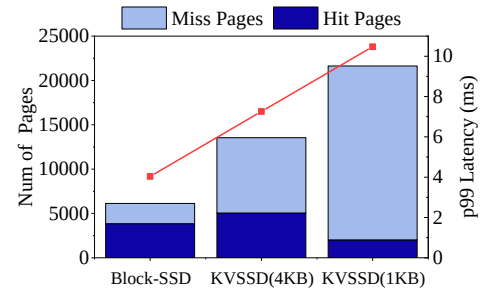


Fig. 4. Average Number of Accessed Mapping Pages and Tail Latencies during KV-SSD GC Process

(OP) ratio of the SSD is 15%. We apply a Zipfian write workload ($\alpha=0.8$) with a 256GB dataset. The write granularity is 4KB for Block-SSD, 4KB and 1KB for KV-SSD, respectively.

Figure 4 illustrates the average number of accessed mapping pages per GC process. The Block-SSD accesses 6,134 mapping pages, achieving a cache hit ratio of 62.7%. In contrast, the KV-SSD incurs substantially more mapping-page accesses (13,544), and the cache hit ratio decreases to 37.3%. This issue is further exacerbated under 1KB accesses due to the increased number of valid KV pairs within an FBG. Consequently, a single GC process involves 21,630 mapping pages, and over 90.7% result in cache misses. This increase in cache misses significantly degrades the performance of GC. Compared to the Block-SSD, the KV-SSD increases the p99 latency by 79.9% and 159.4% under 4KB and 1KB accesses, respectively.

Challenge #3: GC efficiency strongly depends on the mapping hit ratio; as the randomness of hash indexing scatters the mapping entries of massive valid fine-grained KV pairs within each FBG, the mapping updates during GC migration become increasingly inefficient.

Due to physical characteristics of flash memory, data needs to be written sequentially within each FBG. To enhance write parallelism and facilitate flexible data placement, modern SSDs usually maintain multiple active FBGs simultaneously [20], [21], [22]. However, these approaches primarily focus on grouping data with similar lifetimes to minimize GC-induced data migration overhead, rather than mitigating the mapping access overhead during the GC process.

We notice that the efficiency of mapping cache during GC depends heavily on the spatial locality of the mapping entries related to a single FBG. Consequently, when an FBG gathers data with strong mapping locality, the subsequent GC process accesses a much confined set of mapping pages, which inherently improves the efficiency of mapping cache. The support of multiple active FBGs presents a natural opportunity for such data re-aggregation. Specifically, clustering data with adjacent mapping ranges into the same FBG improves the locality of mapping updates and significantly reduces the associated overhead.

Observation #3: Redirecting writes to multiple active FBGs based on mapping ranges enhances mapping locality within each FBG, improving the efficiency of the mapping cache and thereby elevating GC performance.

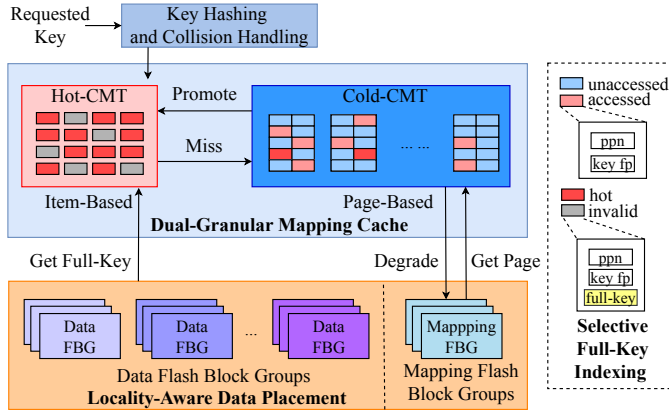


Fig. 5. Overall Architecture of AggreKV

IV. DESIGN

A. Overview of AggreKV

Based on the preceding observations, enhancing the performance of hash-based KV-SSDs requires mitigating mapping inefficiencies across three critical stages: low mapping cache hit ratios caused by fine-grained and randomized hash mapping during address translation, expensive read-before-write forced by key verification during data updates, and severe performance degradation due to massive scattered mapping page accesses during GC. To overcome these mapping-induced challenges, AggreKV implements a multifaceted optimization strategy centered on data locality. Figure 5 presents an architectural overview of AggreKV.

First, to mitigate hash-induced mapping cache misses during address translation (**Challenge #1**), we propose **Proactive Dual-Granular Mapping Cache** (§IV-B). It captures the temporal locality of KV accesses via a fine-grained cache to achieve high space utilization and hit ratios, while maintaining low memory footprint of page-based caching through a coarse-grained cache. Second, to bypass expensive flash-based verifications during updates (**Challenge #2**), we introduce **Selective Full-Key Indexing** (§IV-C). It selectively caches short full keys for frequently updated KV pairs, and prefetches keys upon cache misses to bypass flash key verification in the critical I/O path, thereby enhancing overall update performance. Finally, to resolve inefficient scattered mapping updates during GC (**Challenge #3**), we develop **Locality-Aware Data Placement** (§IV-D). It exploits multiple active FBGs to redirect writes according to mapping entry distributions. This confines the range of mapping page accesses during FBG migrations, significantly improving the GC mapping cache hit ratios and overall efficiency.

B. Proactive Dual-Granular Mapping Cache

In conventional mapping caches, mapping entries are typically managed at the granularity of pages to minimize metadata overhead and exploit the spatial locality of accesses. However, the randomized hash mapping of KV-SSDs inherently disrupts this spatial locality, thereby compromising the storage efficiency of the mapping cache.

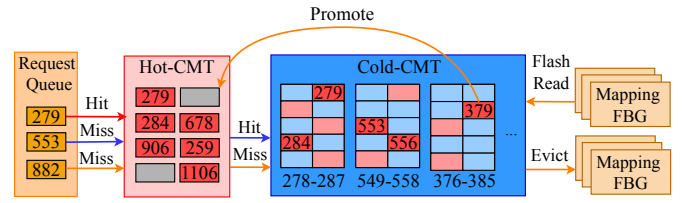


Fig. 6. An Example of Proactive Dual-Granular Mapping Cache

A straightforward solution is to maintain a global mapping cache at fine-grained entry level. This approach decouples hot and cold entries within the same mapping page, thereby preserving cache capacity for hot entries. However, under massive mapping scale of KV-SSDs, simple on-demand caching like LRU or similar policies fails to reliably retain frequently accessed entries, as the limited cache capacity is easily overwhelmed by massive sparse accesses. Furthermore, maintaining cache at a fine granularity incurs substantial metadata overhead in memory. Assuming an 8B hash mapping entry (4B PPN and 4B fingerprint), managing an LRU list at a fine granularity introduces two additional 4B pointers that doubles the size of each entry. Consequently, this overhead reduces the available cache capacity by 50%. Furthermore, fine-grained fetching of mapping entries discards adjacent entries within the same mapping page, thereby sacrificing potential cache hit opportunities.

To address these limitations, we propose **Proactive Dual-Granular Mapping Cache**, designed to **proactively track and exploit** the temporal locality of fine-grained mapping entry accesses, while incurring minimal structural complexity and memory footprint. This design comprises two primary components: a large Cold-CMT that functions as a standard page-based cache to manage flash-resident mapping pages via an LRU policy, and a small Hot-CMT that **proactively** extracts frequently accessed but isolated mapping entries into a memory-efficient hash table. By decoupling these hot entries from their predominantly cold resident pages in the Cold-CMT, the KV-SSD reclaims memory resources to accommodate additional mapping pages, thus improving overall cache efficiency.

Main Workflow. Figure 6 illustrates an example of the **Proactive Dual-Granular Mapping Cache**. Upon receiving a request, the SSD first checks whether the target mapping entry resides in the Hot-CMT. On a cache hit (e.g., Item 279), the SSD directly retrieves the corresponding target PPN. If the Hot-CMT misses the required entry (e.g., Item 553), the SSD searches the Cold-CMT for the associated mapping page to obtain the target mapping entry. If the mapping page is also absent from the Cold-CMT (e.g., Item 882), the SSD initiates a cache replace operation. During this operation, the SSD fetches the mapping page from the flash memory and evicts the coldest mapping page (at the bottom of LRU list) from the Cold-CMT. Besides, the Hot-CMT operates using a write-through policy, where any mapping updates are immediately propagated to the Cold-CMT.

The Cold-CMT periodically promotes hot mapping entries to the Hot-CMT. For example, Item 379 resides in a mapping page where the majority of entries experience infrequent

access. To promote this entry, the SSD employs hashing to locate a candidate slot within the Hot-CMT. When unable to find a suitable slot (i.e., excessive hash collisions), it executes a cache replacement based on the policies introduced in the subsequent section. This replacement does not incur any flash I/O operations, since all entries in the Hot-CMT are clean. Notably, once Item 379 successfully relocates to the Hot-CMT, its source mapping page remains in the Cold-CMT rather than facing immediate eviction. This retention preserves cache hit opportunities for other entries residing in the same page. As the Hot-CMT gradually extracts the active components and the overall access frequency of the page drops, the Cold-CMT eventually evicts the page following the LRU policy.

Caching Policies. The Cold-CMT and Hot-CMT adopt distinct caching policies according to their metadata organization. Because the metadata footprint of the page-based Cold-CMT is relatively low, it employs the conventional LRU policy. However, for the fine-grained Hot-CMT, the cache policy must satisfy two requirements. First, it requires a minimal metadata footprint, as introducing additional fields to an 8B entry incurs significant overhead. Second, it must be able to evaluate and compare the access frequency of cache entries to retain hot mapping entries in the Hot-CMT and promptly evict them once their access frequency declines.

To achieve these objectives, we employ a simple yet effective CLOCK-like strategy that maintains a multi-bit counter for each Hot-CMT entry. Upon insertion, the counter of an entry is initialized to a predefined upper value. Each subsequent hit on the entry increments the counter by one, saturating at a maximum limit. Once a candidate entry is selected for insertion, the Hot-CMT first locate a hash table slot according to the fingerprint field (i.e., `key_fp`). If the slot is occupied, a probing process is initiated to resolve the hash collision. During this process, the counter of each traversed entry is decremented by one. The Hot-CMT overwrites the slot with the candidate entry when the probing encounters either an empty slot or an existing entry with a zero counter. If the probing reaches the maximum collision limit without finding a suitable slot, the Hot-CMT aborts the insertion, assuming that all traversed entries remain hot.

To identify candidates for promotion, we associate a similar counter with each entry in the Cold-CMT. Upon the loading of a mapping page into the Cold-CMT, the counters for all associated entries are initialized to zero. Each subsequent hit to an entry increments the corresponding counter until a maximum value is reached. At regular intervals, the Cold-CMT scans the Top-10% pages within the LRU list to select hot but isolated entries for promotion. An entry qualifies as a hot but isolated entry only if the counter reaches the maximum value and the proportion of entries at the maximum value within the same page is below a predefined threshold (e.g., 70% in our experiments). After scanning, the Cold-CMT attempts to insert all candidate entries into the Hot-CMT, and the counters for all evaluated pages are reset to zero.

C. Selective Full-Key Indexing

Maintaining only fingerprints in the hash mapping table makes the KV-SSD impossible to guarantee data correctness

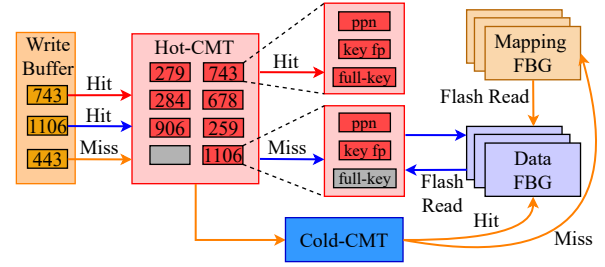


Fig. 7. An Example of Selective Full-Key Indexing

without retrieving the complete key. Fetching the complete key from flash memory incurs negligible overhead for GET () requests, as the complete key is retrieved alongside the requested data during the same flash read operation, and hash collisions rarely occur. For updates, however, the performance penalty becomes substantial due to additional read-before-write. Therefore, efficiently caching the data keys involved in updates while maintaining a minimal memory footprint is crucial to guarantee the write performance of the KV-SSD. AggreKV adopts Selective Full-Key Indexing to address this inefficiency during data updates.

Selective Full-Key Cache. To maximize space efficiency, the full-key cache employs two strategies. First, to avoid adding extra fields to the large-scale global mapping table, we maintain the full-key cache exclusively in the Hot-CMT, where the high access frequency of entries justifies the additional space overhead. As illustrated in Figure 5, a “full-key” field is added to the mapping entries of the Hot-CMT. This field stores an offset within a full-key list, which subsequently points to the actual memory address. Consequently, restricting the total number of cached full keys reduces the length of the full-key list, thereby decreasing the size of the “full-key” field. For instance, by configuring $2^{20} - 1$ full-key cache entries (offset 0 indicates an uncached state), the “full-key” field size can be set to 3B. Assuming that the Hot-CMT occupies 10% of the total memory, the overhead induced by this additional field is merely 3.75%, and the full-key list consumes only 4MB.

Second, to minimize memory footprint while retaining sufficient entries to accelerate updates, the full-key cache employs a selective strategy. To prevent excessively long keys from dominating memory capacity and reducing the total number of cacheable keys, the cache imposes a configurable length limit on admitted keys (e.g., 32B in our evaluations). Furthermore, since key verification during data retrieval does not incur additional flash access overhead, the full-key cache excludes keys from GET () operations. Therefore, insertions into the full-key cache occur exclusively during the flash key verification triggered by updates. For minimal memory overhead, the full-key cache utilizes CLOCK as its replacement strategy.

Figure 7 illustrates the workflow of the selective full-key cache. Assuming that all key verification matches on the first attempt, ❶Item 743 is cached in the Hot-CMT with a valid “full-key” entry, which enables the SSD to directly locate the target key within the full-key cache for rapid verification. ❷Item 1106 is also present in the Hot-CMT but contains an invalid “full-key” entry, thereby requiring the SSD to perform flash-based key verification. ❸Item 443 is absent from the Hot-

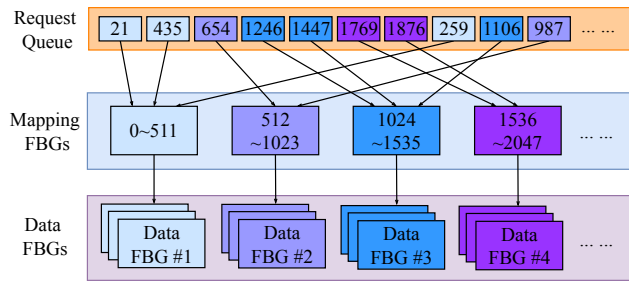


Fig. 8. An Example of Locality-Aware Data Placement

CMT; therefore, the SSD first checks the Cold-CMT. Upon a cache miss, the SSD need to fetch the mapping entry from flash before executing flash-based key verification.

Opportunistic Full-Key Prefetch. KV-SSDs utilize a write buffer to absorb and aggregate incoming KV pairs, as the write granularity of these pairs does not align with the physical size of flash pages. Typically, the mapping lookups and key verification occur only during write buffer flushes, leaving the flash backend idle while the buffer fills. AggreKV advances the mapping lookup and key verification process from the flush phase to the data insertion phase. Therefore, the SSD initiates background indexing and key verification immediately upon data reception. Opportunistically prefetching and verifying the full key effectively masks the overheads of flash-based mapping access and key retrieval for data absent from the full-key cache.

D. Locality-Aware Data Placement

To mitigate the performance degradation of GC stemming from massive dispersed mapping updates, AggreKV employs Locality-Aware Data Placement. It leverages the hardware capability of multiple active FBGs, and redirects write requests into specific FBGs according to the distribution of mapping entries. Upon initialization, the SSD identifies the maximum number of active FBGs. A small number of FBGs are reserved for internal operations, such as the persistence of mapping table and GC migration, while the remaining quota is allocated for write redirection. Specifically, the entire mapping table is divided into partitions based on this available quota. As illustrated in Figure 8, assuming a partition size of 512 entries, the mapping entries ranging from 0 to 511 are assigned to Partition #1, the entries ranging from 512 to 1023 are assigned to Partition #2, and so forth.

The SSD maintains at least one active FBG for each partition, dedicated to receiving data whose mapping entries fall within the corresponding mapping range. For instance, as shown in Figure 8, the requests in the queue arrive sequentially from left to right, but their payloads are dispatched to different destinations based on the partitions of the corresponding mapping entries: {21, 435, 259} are written to Data FBG #1, {654, 987} to Data FBG #2, and {1246, 1447, 1106} to Data FBG #3. Consequently, the mapping entries related to each Data FBG are effectively confined to a specific range within the mapping table. During GC, this confinement drastically reduces the number of accessed mapping pages, ensuring

a higher hit ratio of mapping cache, thereby significantly enhancing the overall performance of GC.

FBGs for each partition are allocated on-demand. When the flash backend reaches its capacity, GC is invoked to reclaim free blocks. Conventional FTL designs select victim FBGs by evaluating metrics such as the number of valid pages, write frequency, and block wear levels [23]. AggreKV largely inherits these principles but integrates the target write partition into the decision-making process. When a victim FBG belongs to the same partition as the incoming write request, the SSD simply migrates the valid pages to the reserved FBG and recycles the erased victim FBG to serve as the new reserved FBG. Conversely, if the victim FBG resides in a different partition, the mechanism differs to ensure that the target partition secures an active FBG without compromising the globally reserved FBG. In this scenario, the partition containing the victim FBG must provide two empty FBGs after GC, one of which replaces the global reserved FBG, while the other is reassigned to the target partition. Consequently, this requirement adapts the heuristic for victim selection. For example, rather than merely identifying the FBG with the fewest valid pages, the SSD evaluates the cumulative migration cost across partitions to determine the optimal victims.

E. Workload-Adaptive Cache Space Allocation

In AggreKV, the on-board DRAM cache is shared among three components: the Hot-CMT, the Cold-CMT, and the full-key cache. Since real-world workloads vary in read/write ratios and access skewness, static memory partitioning inevitably leads to resource underutilization. To maximize overall efficiency, AggreKV dynamically allocates their capacities based on real-time workload characteristics: the Hot-CMT captures temporal locality for highly skewed reads, the full-key cache bypasses flash verifications for intensive updates, and the Cold-CMT provides foundational page-level mapping coverage.

Specifically, the capacity of the Hot-CMT is highly correlated with temporal locality. To expand the Hot-CMT, AggreKV monitors the entries evicted from Hot-CMT. If a significant proportion of evicted entries (i.e., over 50%) possess a high hotness counter value, it indicates the active working set outgrows the current Hot-CMT capacity. In this case, AggreKV scales up the Hot-CMT by a step size (i.e., 1%) to retain these highly active entries. Conversely, to shrink the Hot-CMT, AggreKV also tracks the total number of highly active entries in the Hot-CMT. When this number falls below a threshold (i.e., 25%), it implies cache underutilization. Consequently, AggreKV shrinks the Hot-CMT by a step size, returning the memory resources to the global budget, thereby maximizing overall memory utilization.

Concurrently, the memory distribution between the full-key cache and the Cold-CMT is governed by lightweight ghost caches. AggreKV tracks the potential gain of expanding the full-key cache via a ghost key cache (i.e. a CLOCK cache storing only Hot-CMT slot numbers of recently evicted keys), and measures the hit loss of shrinking it by monitoring the tail window of the CLOCK pointer. By comparing these metrics with the expansion gains and shrinkage losses monitored by

the Cold-CMT's general LRU ghost cache, AggreKV transfers a memory step to the full-key cache only when its expansion gain strictly outweighs the Cold-CMT's shrinkage loss. Otherwise, the capacity is dynamically reclaimed by the Cold-CMT. The metadata overhead of both ghost caches is minimal, because they only track entry/page metadata within a single memory step. This policy establishes continuous balance that maximizes overall access efficiency.

F. Analysis of Metadata Overhead

Hot-CMT. Within each Hot-CMT entry, we introduce a multi-bit counter to indicate access frequency and a “full-key” field to store the full-key cache offset. The counter size is set to 2 bits, and the size of the “full-key” field is 3B. The additional memory footprint of these two fields depends on the size of the Hot-CMT. For example, a Hot-CMT configured as 10% of the total cache capacity will account for 4.06% of the metadata footprint.

Cold-CMT. We similarly introduce a multi-bit counter into each Cold-CMT entry and allocate 2 bits for its size. Aside from this addition, the Cold-CMT metadata is identical to that of a conventional page-based mapping cache. Consequently, incorporating the counter increases the global memory footprint by merely 2.81%.

Full-Key Cache. The full-key cache contains a full-key list, a reference bitmap for CLOCK algorithm, and a memory region for storing complete key strings. In our experiments, we cap the maximum number of cached keys at $2^{20} - 1$. The full-key list maintains a 4B memory pointer for each key, which limits its overall size to 4MB, and the reference bitmap consumes only 128KB.

V. EVALUATION

A. Experiment Setup

To construct the experimental platform, we implement AggreKV on a custom user-space simulator adapted from the backend flash model of FEMU [24], an open-source NVMe SSD emulator that enables accurate and flexible simulation of various SSD architectures. The emulated backend flash provides a logical storage capacity of 256GB with an overprovisioning (OP) ratio of 15%, and it contains 4 channels, each connecting 8 flash dies. Each die consists of 1,178 flash blocks, and each block comprises 2,048 pages of 4KB size. The flash read, program, and erase latencies are configured at 50 μ s, 500 μ s, and 5000 μ s, respectively. The capacity of the on-board DRAM cache is configured at 256MB.

To comprehensively evaluate the performance of the proposed AggreKV scheme for KV-SSDs, we compare it against four baseline configurations: a conventional Block-SSD managed by a host-side KV store (RocksDB[1]), a classic DFTL hash-based KV-SSD, a state-of-the-art hash-based KV-SSD known as RHIK[8], and an LSM-tree-based KV-SSD. The evaluation of the Block-SSD is conducted by replaying block-level I/O traces collected from the host-side execution. Table I summarizes these configurations.

For the Block-SSD, the on-board DRAM is able to retain the entire 256MB L2P mapping table. For hash-based KV-SSD

TABLE I
COMPARED KV-SSD SETUPS IN EVALUATION

Setup	Description
Block	Conventional Block-SSD under host-side KV store RocksDB[1]
Hash	Classic hash-based KV-SSD baseline adopting DFTL mapping page cache management
LSM-tree	KV-SSD baseline with LSM-tree indexing and background data maintenance. The mapping segment is cached on-demand
RHIK	State-of-the-art hash-based KV-SSD with a two-level reconfigurable-sized hash index
AggreKV	Our hash-based KV-SSD design optimized with dual-granularity cache, selective full-key indexing and locality-aware data placement

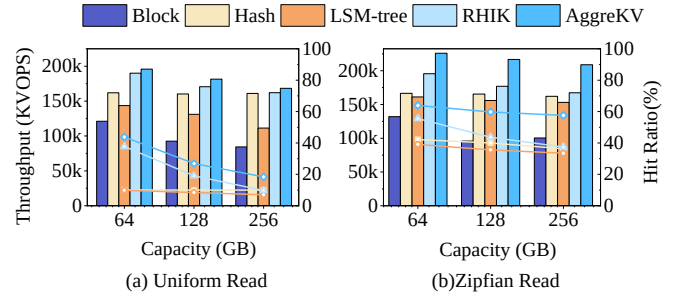


Fig. 9. Throughput and Mapping Hit Ratio under Uniform and Zipfian Read Workloads

schemes (i.e., Hash, RHIK and AggreKV), the fingerprint field size is 4B. The Hash baseline fixes the mapping table size to accommodate a maximum dataset of 256GB comprising 1KB KV pairs with a load factor of 0.75, whereas RHIK and AggreKV dynamically scale the mapping table according to the dataset size up to this limit. For AggreKV, the Cold-CMT triggers promotion to the Hot-CMT every 500 times of cache eviction, and the adaptive cache space allocation is evaluated every 100k requests. The default number of active FBGs for locality-aware data placement is 8.

In the subsequent sections, we systematically evaluate AggreKV through comprehensive experiments measuring its read and update performance under diverse random access patterns (i.e., uniform and Zipfian with $\alpha=1.0$) and working set capacities (i.e., from 64GB to 256GB), evaluating its garbage collection efficiency under different active FBG numbers, benchmarking overall throughput via YCSB workloads [25], analyzing its sensitivity to the key length distribution and measuring its CPU overhead. Unless otherwise specified, the size of KV pairs is 1KB, and the key length is set to 32B.

B. Read Performance Analysis

Figure 9 presents the read throughput and mapping cache hit ratio of the schemes under uniform and Zipfian read workloads. Under uniform read workload, LSM-tree exhibits poor performance because it requires a top-down scan of the level list, resulting in additional mapping I/O. The KV store over Block-SSD suffers from the overhead of the host storage stack, further amplifies the I/O size and exacerbates this mapping access inefficiency. AggreKV outperforms other schemes with constrained improvements. Due to the lack of

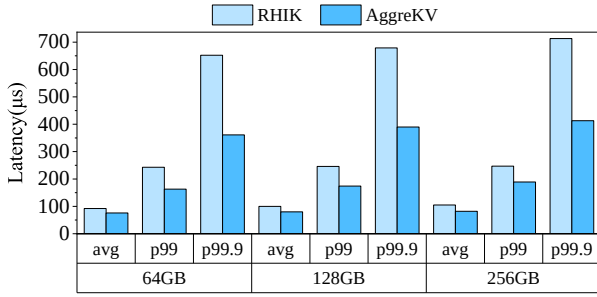


Fig. 10. Average- and Tail Latencies under Zipfian Read Workloads

access skewness, there is no chance to aggregate hot KV pairs using the Hot-CMT. However, compared to Hash and RHIK, AggreKV still exposes 19.69% and 7.50% cache hit ratio improvement, 12.80% and 3.90% performance gains on average, respectively. Because the slot count of hash table usually exceeds the number of actual KV pairs (i.e., load factor), empty slots inevitably exist in most scenarios. Consequently, even without a strong data access skew, the fine-grained Hot-CMT can still filter out these empty slots from the mapping pages. This reduces cache space occupation and enhances the overall cache utilization.

Under Zipfian read workload, AggreKV efficiently captures hot but isolated mapping entries in the hash table, thereby avoiding the sharp performance degradation that affects other schemes relying on page-based cache mechanisms. Among three working set capacities, AggreKV achieves an average improvement of 20.01% on the mapping cache hit ratio and provides a 30.47% throughput enhancement over three KV-SSD baselines. At 64GB, AggreKV improves the hit ratio and read throughput by 8.31% and 13.42%, compared to RHIK. In the 256GB scenario, this hit ratio improvement further reaches 20.13%, which leads to a 24.91% improvement on throughput. These results demonstrate that AggreKV efficiently exploits the temporal locality of hot accesses under Zipfian workload, maintaining excellent cache utilization and read performance even under fully loaded conditions.

Notably, as the working set capacity increases from 64GB to 256GB, the mapping cache hit ratio of RHIK degrades dramatically by 28.02% and 18.14% under uniform and Zipfian workload, respectively. This degradation occurs because RHIK utilizes a dynamical-sized hash table. For small working sets, the table remains compact, allowing a substantial portion of the hash table to reside within the cache. However, as the capacity of the working set increases, the hash table expands accordingly, which leads to a significant reduction in the cache hit ratio. In contrast, AggreKV efficiently extracts useful mapping entries even when the mapping table becomes extremely large, thereby maintaining high cache efficiency.

We also evaluate the average and tail latency of RHIK and AggreKV under Zipfian read workloads. As Figure 10 illustrates, at a working set capacity of 64GB, AggreKV reduces the average latency from 92µs to 76µs, achieving a 17.4% reduction. At 256GB, the average latency decreases from 105µs to 82µs, representing a 21.9% reduction. Regarding tail latency, AggreKV demonstrates more substantial improvements. Across all tested working set capacities, AggreKV

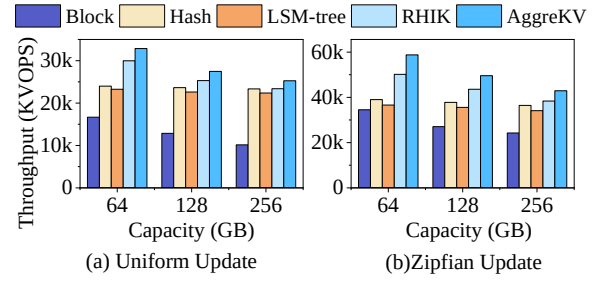


Fig. 11. Throughput under Uniform and Zipfian Update Workloads

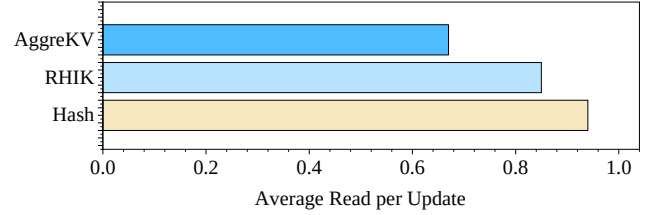


Fig. 12. Average Flash Reads under Zipfian Workload

decreases the p99 and p99.9 latency by up to 33% and 45%, respectively. These improvements effectively mitigate long-tail latency spikes and significantly enhance latency predictability.

Conclusion #1: The proactive dual-granular mapping cache reduces cache waste under uniform workloads and captures data hotness under skewed workloads, thereby significantly improving the cache hit ratio and read throughput while significantly reducing average- and tail latencies.

C. Update Performance

To avoid overhead of flash-based key verification before every update, AggreKV adopts the selective full-key cache strategy. Figure 11 presents the update throughput under both uniform and Zipfian workloads. Under uniform update scenario, AggreKV improves the update throughput by 8.05%–9.63%, compared to RHIK. Under Zipfian update scenario, which exhibits a stronger access skew, this improvement increases to 11.83%–17.14%. The LSM-tree scheme demonstrates lower update performance, as the background compaction incurs substantial I/O and sorting computation overhead. Similarly, the KV store over Block-SSD performs the worst due to the combined overhead of background compaction and the host storage stack. Compared to other KV-SSD baselines, AggreKV achieves an average improvement of 23.95% in update throughput across all workloads.

Notably, under Zipfian workloads, the performance gain of AggreKV over RHIK is most pronounced at 64GB (17.14%). This is because a smaller working set capacity yields a higher cache hit ratio. Consequently, the elimination of flash verification reads for frequently updated KV pairs constitutes a larger proportion of the overall update performance. However, although at a large working set capacity of 256GB, AggreKV still exhibits highest throughput, demonstrating the effectiveness of the selective full-key cache strategy.

Furthermore, we analyze the average number of flash reads per update under the 64GB Zipfian workload, as presented in Figure 12. Among all schemes, AggreKV achieves the lowest

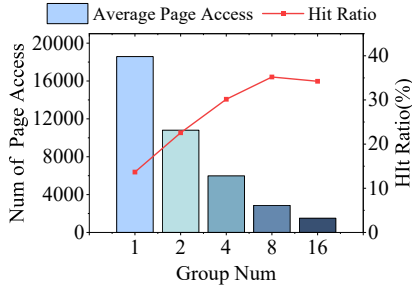


Fig. 13. Average Access Number and Cache Hit Ratio of Mapping Pages per GC Process

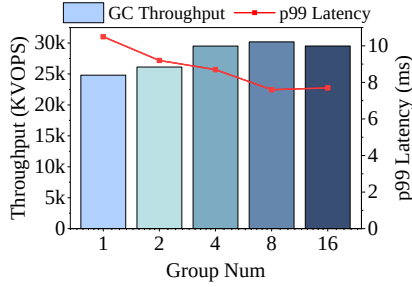


Fig. 14. Update Throughput and Tail Latencies of AggreKV under Different Number of Active FBGs

average number of flash reads (0.67), which is 21.18% lower than RHIK (0.85) and 28.72% lower than Hash (0.94). This significant reduction in flash read operations directly stems from the selective full-key cache strategy and the fine-grained Hot-CMT design of AggreKV, which effectively eliminate flash reads for mapping and verification prior to updates.

Conclusion #2: The selective full-key indexing effectively reduces the average number of flash reads per update and consistently improves update throughput. This performance gain is more pronounced under skewed workloads with higher cache hit ratios.

D. Garbage Collection Efficiency and Wear Leveling

To reduce the number of mapping pages accessed during GC, AggreKV employs locality-aware data placement utilizing multiple active FBGs. The experimental results of Zipfian update workloads presented in Figure 13 and 14 demonstrate the effectiveness of this design. The working set capacity is configured at 256GB.

When operating with a single FBG (i.e., $N=1$), AggreKV works the same as Hash and RHIK. Consequently, the KV-SSD accesses 18,581 mapping pages on average during a single GC process, which yields a GC mapping cache ratio of 13.67%. As the active FBGs increases, the number of mapping pages accessed during GC is significantly reduced. With 8 active FBGs ($N=8$), the mapping pages associated with a single FBG declines by 84.59%. Consequently, the GC mapping cache hit ratio increases to 35.19%, leading to a 21.72% growth of throughput and a 27.34% decline of tail latency. This improvement confirms that the locality-aware data placement effectively mitigates the GC-induced I/O overhead.

However, enabling more active FBGs incurs higher overhead in metadata management and additional memory re-

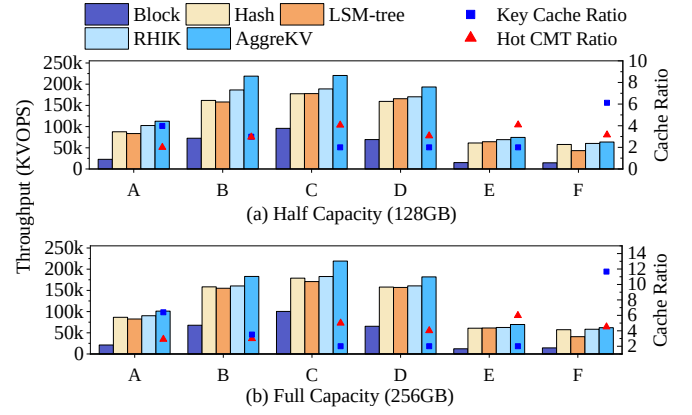


Fig. 15. Throughput and Cache Ratios under YCSB Workloads at 128GB and 256GB Working Set Capacity

sources (e.g., write buffer). This potentially reduces the usable mapping cache capacity, thereby degrading the efficiency of address translation. As Figure 14 illustrates, the update KVOPS of AggreKV rises from 24,803 ($N=1$) to a peak of 30,190 ($N=8$), and then declines to 29,517 ($N=16$). Therefore, a moderate number of FBGs (e.g., $N=8$) is preferable for improving the cache hit ratio and enhancing update performance.

Furthermore, we evaluate the impact of the locality-aware data placement on SSD endurance by scaling the logical capacity to 64GB (15% OP) with a 48GB working set. After writing 14TB of data under a Zipfian workload ($\alpha = 0.8$), AggreKV strictly preserves wear leveling. The variance of data FBG erase counts is merely 0.47 (vs. 2.07 in RHIK), with a negligible maximum difference of 7. Notably, AggreKV maintains a comparable average erase count per FBG to the baseline (664 vs. 660 in RHIK). This indicates that despite the potential for partition-level write skew, the globally shared reserved FBGs for GC effectively distribute the localized wear. Consequently, AggreKV preserves the SSD wear leveling and ensures overall device endurance.

Conclusion #3: The locality-aware data placement substantially reduces mapping page read overhead during GC without compromising the SSD lifespan. By carefully configuring the number of active FBGs, it improves the GC efficiency and contributes to the overall update performance.

E. Overall Performance

We further evaluate the overall performance of AggreKV under six YCSB workloads using two working set capacities corresponding to half (128GB) and full (256GB) disk capacities. As illustrated in Figure 15, AggreKV consistently achieves higher throughput than other schemes across all six workloads.

With a working set capacity of 128GB, compared to other KV-SSD baselines, the performance gain of AggreKV range from 14.84% to 30.56%, yielding an average improvement of 21.60%. The most significant gain is observed in the read-only workload YCSB C, while the performance improvements for the other workloads remain stable within a narrow margin.

As the working set increases to 256GB, the advantages of AggreKV remain pronounced, although the improvement

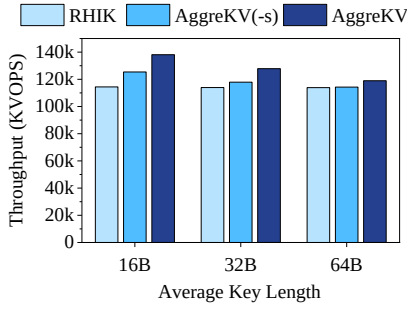


Fig. 16. Throughput under YCSB A workload with Different Key Length Distributions

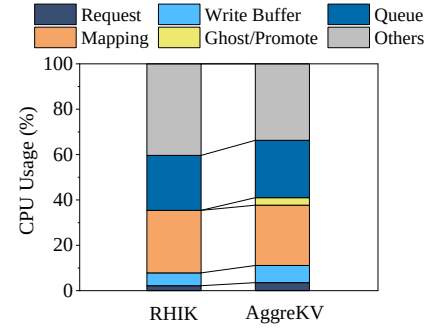


Fig. 17. CPU Usage Breakdown of FTL Thread

margins slightly narrow due to the intensified cache pressure, ranging from 12.89% to 23.59% (17.72% on average). Similar to the 128GB scenario, the read-only workload YCSB C continues to achieve the highest gain. These results demonstrate that AggreKV effectively mitigates performance bottlenecks of hash-based KV-SSD with on-device memory constraints, providing higher throughput compared to other schemes.

As observed, the allocation of the Hot-CMT and the full-key cache adapts to the distinct I/O characteristics of each workload. For update-heavy workloads (A and F), the mechanism automatically assigns a higher proportion of memory to the full-key cache to bypass the costly flash-based key verification during frequent data updates. Conversely, for read-intensive workloads (B, C, D, and E), the mechanism adaptively shrinks the full-key cache size and allocates more resources to the Hot-CMT to achieve higher mapping hit ratio. This dynamic resource provisioning ensures that AggreKV consistently achieves optimal performance across diverse access patterns.

Conclusion #4: AggreKV successfully mitigates the mapping-induced performance bottlenecks inherent in hash-based KV-SSDs. By dynamically adapting cache proportions to varying workload characteristics, AggreKV consistently demonstrates the best performance compared to the other schemes across all six YCSB workloads.

F. Sensitivity Analysis on Key Length Distribution

To evaluate the impact of key length variations, we run the YCSB A workload with key lengths following a Gaussian distribution ($\mu \in \{16B, 32B, 64B\}$, $\sigma = 0.15$). We compare RHIK, AggreKV(-s) (which attempts to caches all keys regardless of type or length), and the default AggreKV. The results are illustrated in Figure 16.

Overall, both AggreKV variants consistently outperform the baseline RHIK across all key length distributions. However, as the average key length increases, the AggreKV variants experience a performance degradation. This decline is driven by two factors: first, longer keys consume a larger memory footprint, naturally reducing the total number of keys that can be cached; second, the default AggreKV actively bypasses excessively long keys, forcing it to fall back to flash-based verification.

Despite this degradation, AggreKV consistently outperforms AggreKV(-s) across all distributions. At $\mu=16B$ and 32B, the selective variant achieves throughput gain of 10.08%

and 8.41% over the non-selective approach. Even at $\mu=64B$, it maintains a higher throughput (118,946 KVOPS) than the non-selective variant (114,288 KVOPS). This confirms the rationality of our selective admission policy: by filtering out read requests and bypassing oversized keys, AggreKV avoids wasting precious cache capacity and strictly preserves the limited memory budget for the most critical, update-intensive short keys.

G. CPU Overhead Analysis

To evaluate the computational overhead introduced by the design of AggreKV, we breakdown and analyze the CPU execution cycles of both RHIK and AggreKV under the YCSB A workload. As illustrated in Figure 17, the overall CPU usage is categorized into processing incoming requests (Request), managing the write buffer (Write Buffer), handling K2P mapping translations and cache maintenance (Mapping), processing internal I/O queues (Queue), and AggreKV-specific operations including Ghost cache management and entry promotion (Ghost/Promote).

Notably, the newly introduced Ghost/Promote operations in AggreKV consume a mere 2.9% of the total CPU cycles, proving that the dual-granular cache management and promotion policies of AggreKV are highly lightweight. Furthermore, the CPU overhead for Mapping actually decreases from 26.4% in RHIK to 23.2% in AggreKV. This reduction indicates that the computational cost of managing the fine-grained Hot-CMT is offset by the improved cache hit ratio, which mitigates CPU-intensive cache miss handling and flash I/O processes. Although the Request and Write Buffer overheads experience a negligible increase (1.0% and 1.2%, respectively) due to the selective full-key indexing and locality-aware placement logic, the overall computational footprint remains highly efficient. In conclusion, AggreKV introduces minimal CPU overhead, making it practical for modern KV-SSD controllers with limited computing resources.

VI. RELATED WORKS

A. KV-SSD Index Designs

Existing research optimizes KV-SSD designs along LSM-tree-based or hash-based approaches. For LSM-tree-based KV-SSDs, LightStore improves power efficiency through a lightweight FTL and software adapters [26]. iLSM-SSD reduces the overhead of compaction and garbage collection via

intra-device key-value separation and scattered logging [27]. PinK bounds the worst-case latency by pinning top-level indices in DRAM and offloading compaction to hardware accelerators [4]. IterKVSSD improves the scan throughput of range queries by leveraging internal SSD parallelism for index and value prefetching [28]. AnyKey minimizes the flash-spill of metadata and I/O amplification by integrating sorted data segment groups with in-DRAM hash lists and a value log [19].

For hash-based KV-SSDs, KAML establishes an early baseline by combining a multi-log architecture with host-side key conversion [5]. The KV-SSD developed by Samsung expands command support and directly indexes user keys to flash addresses [6]. KVFTL introduces partition-based slicing to resolve internal fragmentation caused by variable-length values [29]. Furthermore, EMT adopts full-page partitions with a single fragment buffer to reduce metadata memory overhead and enhance read performance [7]. RHIK implements a dynamically resizable hash structure to handle varying dataset cardinalities [8]. Distinct from these previous studies, AggreKV focuses on the degradation of mapping efficiency under large-scale datasets, which typically results from a lack of awareness regarding data access locality.

B. KV-SSD-Oriented Research

Recent studies also explore host-device co-design to extend the capabilities of KV-SSDs and optimize the software stack for system efficiency. For example, compound commands are proposed to reduce I/O overhead and supports atomic transactions by coalescing small operations into a single request [22]. KVMD accelerates failure recovery for variable-length objects through size-aware redundancy techniques [30]. KVMRAID mitigates write amplification in multi-device arrays by employing an update-friendly erasure coding scheme [31]. KEVIN removes host-side journaling overhead by offloading file indexing and transaction management to the storage device [32]. Dotori addresses the functional limitations of KV-SSDs in database applications by introducing an append-only tree structure that persists index updates as independent logs [33]. Furthermore, KVRangeDB facilitates efficient range queries on hash-based KV-SSDs by combining key-value separation with a compact LSM-tree index [34].

VII. CONCLUSION

In this paper, we propose AggreKV, a locality-centric K2P mapping approach that mitigates mapping-induced performance bottlenecks in hash-based KV-SSDs. By systematically reconstructing data locality, AggreKV resolves hash mapping inefficiencies under constrained on-device DRAM. Specifically, a **proactive** dual-granular mapping cache extracts hot entries to capture temporal locality, maximizing hit ratios with minimal metadata overhead. Additionally, selective full-key indexing bypasses expensive read-before-write flash verifications during updates by caching short complete keys. Finally, locality-aware data placement clusters data from adjacent mapping regions into shared FBGs, confining mapping updates to enhance garbage collection efficiency. Extensive

evaluations demonstrate that AggreKV outperforms state-of-the-art designs with negligible hardware overhead, enhancing read and update throughput by **30.47%** and **23.95%**, while reducing tail latencies by up to **45%** and 27.34%, respectively.

REFERENCES

- [1] "Facebook/rocksdb," Meta, accessed: Feb. 5, 2026. [Online]. Available: <https://github.com/facebook/rocksdb>
- [2] M. P. Saha, A. Maruf, B. S. Kim, and J. Bhimani, "KV-SSD: What Is It Good For?" in *2021 58th ACM/IEEE Design Automation Conference (DAC)*, Dec. 2021, pp. 1105–1110.
- [3] S.-M. Wu, K.-H. Lin, and L.-P. Chang, "KVSSD: Close integration of LSM trees and flash translation layer for write-efficient KV store," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, Mar. 2018, pp. 563–568.
- [4] J. Im, J. Bae, C. Chung, Arvind, and S. Lee, "PinK: High-speed In-storage Key-value Store with Bounded Tails," in *2020 USENIX Annual Technical Conference (USENIX ATC)*, 2020, pp. 173–187.
- [5] Y. Jin, H.-W. Tseng, Y. Papakonstantinou, and S. Swanson, "KAML: A Flexible, High-Performance Key-Value SSD," in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb. 2017, pp. 373–384.
- [6] Y. Kang, R. Pitchumani, P. Mishra, Y.-s. Kee, F. Londono, S. Oh, J. Lee, and D. D. G. Lee, "Towards building a high-performance, scale-in key-value storage system," in *Proceedings of the 12th ACM International Conference on Systems and Storage (SYSTOR)*. New York, NY, USA: Association for Computing Machinery, May 2019, pp. 144–154.
- [7] T. Chang, J.-W. Hsieh, T.-C. Chang, and L.-W. Lai, "EMT: Elegantly Measured Tanner for Key-Value Store on SSD," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 1, pp. 91–103, Jan. 2022.
- [8] M. P. Saha, B. S. Kim, H. S. Gunawi, and J. Bhimani, "RHIK: Re-configurable Hash-based Indexing for KVSSD," in *Proceedings of the 32nd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*. New York, NY, USA: Association for Computing Machinery, Aug. 2023, pp. 319–320.
- [9] A. Gupta, Y. Kim, and B. Ugaonkar, "Df1l: a flash translation layer employing demand-based selective caching of page-level address mappings," *SIGPLAN Not.*, vol. 44, no. 3, p. 229–240, Mar. 2009.
- [10] Y. Zhou, F. Wu, P. Huang, X. He, C. Xie, and J. Zhou, "An efficient page-level FTL to optimize address translation in flash memory," in *Proceedings of the Tenth European Conference on Computer Systems*. Bordeaux France: ACM, Apr. 2015, pp. 1–16.
- [11] B. Lepers, O. Balmau, K. Gupta, and W. Zwaenepoel, "KVell: The design and implementation of a fast persistent key-value store," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*. New York, NY, USA: Association for Computing Machinery, Oct. 2019, pp. 447–461.
- [12] R. L. Rivest, "The MD5 Message-Digest Algorithm," Internet Engineering Task Force, Request for Comments RFC 1321, Apr. 1992.
- [13] T. Hansen and D. E. Eastlake 3rd, "US Secure Hash Algorithms (SHA and HMAC-SHA)," Internet Engineering Task Force, Request for Comments RFC 4634, Aug. 2006.
- [14] Wikipedia, "Linear probing," accessed: Feb. 5, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Linear_probing&oldid=1319260812
- [15] R. Chen, Z. Qin, Y. Wang, D. Liu, Z. Shao, and Y. Guan, "On-demand block-level address mapping in large-scale nand flash storage systems," *IEEE Transactions on Computers*, vol. 64, no. 6, p. 1729–1741, Jun. 2015.
- [16] Y. Wang, Z. Qin, R. Chen, Z. Shao, and L. T. Yang, "An adaptive demand-based caching mechanism for nand flash memory storage systems," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 22, no. 1, Dec. 2016. [Online]. Available: <https://doi.org/10.1145/2947658>
- [17] L. Marmol, S. Sundararaman, N. Talagala, and R. Rangaswami, "NVMKV: A scalable, lightweight, FTL-aware Key-Value store," in *2015 USENIX Annual Technical Conference (USENIX ATC)*, Jul. 2015, pp. 207–219.
- [18] Samsung Electronics, "Kv-ssd firmware introduction," accessed: Feb. 5, 2026. [Online]. Available: https://github.com/OpenMPDK/KVSSD/wiki/presentation/kvssd_seminar_2018/kvssd_seminar_2018_fw_introduction.pdf

- [19] C. Park, J. Lee, C.-Y. Liu, K. Kang, M. T. Kandemir, and W. Choi, "Anykey: A key-value ssd for all workload types," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. New York, NY, USA: Association for Computing Machinery, Mar. 2025, p. 47–63.
- [20] J.-U. Kang, J. Hyun, H. Maeng, and S. Cho, "The multi-streamed solid-state drive," in *USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2014.
- [21] J. Yang, R. Pandurangan, C. Choi, and V. Balakrishnan, "Autostream: automatic stream management for multi-streamed ssds," in *Proceedings of the 10th ACM International Systems and Storage Conference (SYSTOR)*. New York, NY, USA: Association for Computing Machinery, May 2017, pp. 1–11.
- [22] T. Kim, D. Hong, S. S. Hahn, M. Chun, S. Lee, J. Hwang, J. Lee, and J. Kim, "Fully automatic stream management for multi-streamed ssds using program contexts," in *17th USENIX Conference on File and Storage Technologies (FAST 19)*, 2019, pp. 295–308.
- [23] S. Oh, J. Kim, S. Han, J. Kim, S. Lee, and S. H. Noh, "MIDAS: Minimizing Write Amplification in Log-Structured Systems through Adaptive Group Number and Size Configuration," in *22nd USENIX Conference on File and Storage Technologies (FAST 24)*, 2024, pp. 259–275.
- [24] H. Li, M. Hao, M. H. Tong, S. Sundararaman, M. Björling, and H. S. Gunawi, "The CASE of FEMU: cheap, accurate, scalable and extensible flash emulator," in *16th USENIX Conference on File and Storage Technologies (FAST 18)*, 2018, pp. 83–90.
- [25] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with ycsb," in *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*. New York, NY, USA: Association for Computing Machinery, Jun. 2010, pp. 143–154.
- [26] C. Chung, J. Koo, J. Im, Arvind, and S. Lee, "Lightstore: Software-defined network-attached key-value drives," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. New York, NY, USA: Association for Computing Machinery, Apr. 2019, p. 939–953.
- [27] C.-G. Lee, H. Kang, D. Park, S. Park, Y. Kim, J. Noh, W. Chung, and K. Park, "iLSM-SSD: An Intelligent LSM-Tree Based Key-Value SSD for Data Analytics," in *2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, Oct. 2019, pp. 384–395.
- [28] S. Lee, C.-G. Lee, D. Min, I. Park, W. Chung, A. Sivasubramaniam, and Y. Kim, "Iterator Interface Extended LSM-tree-based KVSSD for Range Queries," in *Proceedings of the 16th ACM International Conference on Systems and Storage (SYSTOR)*. New York, NY, USA: Association for Computing Machinery, Jun. 2023, pp. 60–70.
- [29] Y.-T. Chen, M.-C. Yang, Y.-H. Chang, T.-Y. Chen, H.-W. Wei, and W.-K. Shih, "Co-Optimizing Storage Space Utilization and Performance for Key-Value Solid State Drives," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 38, no. 1, pp. 29–42, Jan. 2019.
- [30] R. Pitchumani and Y.-s. Kee, "Hybrid data reliability for emerging key-value storage devices," in *18th USENIX Conference on File and Storage Technologies (FAST 20)*, 2020, pp. 309–322.
- [31] M. Qin, A. L. N. Reddy, P. V. Gratz, R. Pitchumani, and Y. S. Ki, "KVRAID: High performance, write efficient, update friendly erasure coding scheme for KV-SSDs," in *Proceedings of the 14th ACM International Conference on Systems and Storage (SYSTOR)*. New York, NY, USA: Association for Computing Machinery, Jun. 2021, pp. 1–12.
- [32] J. Koo, J. Im, J. Song, J. Park, E. Lee, B. S. Kim, and S. Lee, "Modernizing file system through in-storage indexing," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021, pp. 75–92.
- [33] C. Duffy, J. Shim, S.-H. Kim, and J.-S. Kim, "Dotori: A Key-Value SSD Based KV Store," *Proceedings of the VLDB Endowment*, vol. 16, no. 6, pp. 1560–1572, Feb. 2023.
- [34] M. Qin, Q. Zheng, J. Lee, B. Settlemeyer, F. Wen, N. Reddy, and P. Gratz, "KVRangeDB: Range Queries for a Hash-based Key-Value Device," *ACM Transactions on Storage*, vol. 19, no. 3, pp. 24:1–24:21, Jun. 2023.